S/W Engineering Final Lab Report Guidelines

Table of Contents

Appendices

Appendix A: Full Source Code Snapshots & Repository Link

Appendix B: Complete Test Cases (Detailed Tables)

Appendix C: Sample Screenshots (if needed) of the Working System

Appendix D: Meeting Minutes / Progress Logs

Appendix E: Additional Diagrams (Deployment Diagram, Component Diagram, etc.)

Appendix F: AI Model Documentation / Training Details (If Applicable)

Explanation

1. Introduction

- **1.1 Goals and Objectives of the Project** — List the main goals and specific measurable objectives of your project.
- **1.2 Scope of the Work**
  - **1.2.1 Current Situation and Context** — Describe the existing problem or manual system and why a new system is needed.
  - **1.2.2 Competing Products (Available in Market)** — Mention similar existing software and their limitations.
- **1.3 System Overview** — Give a high-level description of what your system does, its main features, and target users.
- **1.4 Structure of the Document** — Briefly explain what each chapter of the report contains.
- **1.5 Terms, Acronyms, and Abbreviations Used** — Define all technical terms, acronyms, and abbreviations used in the report.

2. Project Management Plan

- **2.1 Project Organization**
  - **2.1.1 Individual Contribution to the Project** — Detail what each team member contributed.
- **2.2 Process Model Used**
  - **2.2.1 Rationale for Choosing Lifecycle Model** — Mention the model (e.g., Waterfall, Agile, Spiral) and justify why you chose it.
- **2.3 Risk Analysis** — Identify potential risks and their mitigation strategies.
- **2.4 Constraints to Project Implementation** — Mention technical, time, resource, or budget constraints.
- **2.5 Hardware and Software Resources (Tools/Languages) Requirements** — List required hardware, software, IDEs, languages, and frameworks.

- **2.6 Project Timeline and Schedule** — Include a Gantt chart or timeline with phases and milestones.
- **2.7 Estimated Budget** — Breakdown of estimated costs (tools, hosting, etc.).
- **2.8 Social/Cultural/Environmental Impact of the Project** — Discuss positive/negative impacts.

3. Requirement Specifications

- **3.1 Stakeholders for the System** — List all stakeholders and their roles.
- **3.2 Use Case Diagram with Graphical and Textual Description** — Diagram + detailed description of each use case (actors, preconditions, postconditions, steps).
- **3.3 Activity Diagram** — Show workflow of major processes.
- **3.4 Static Model – Class Diagram** — Show classes, attributes, methods, and relationships.
- **3.5 Dynamic Model – Sequence Diagram** — Show interaction between objects for key scenarios.
- **3.6 Safety and Security Requirements**
  - **3.6.1 Access Requirements** — Authentication, authorization, roles.
  - **3.6.2 Integrity Requirements** — Data accuracy and consistency.
  - **3.6.3 Privacy Requirements** — Data protection and privacy measures.

4. Architecture

- **4.1 Architectural Model/Style Used**
  - **4.1.1 Rationale for Choosing the Architectural Model/Style** — Explain choice (e.g., MVC, Client-Server, Layered) and why it fits.
- **4.2 Architectural Design (High-Level Design)** — Overall system architecture diagram, layers, components, and their interactions.
- **4.3 Technology, Software, and Hardware Used** — Detailed tech stack justification.
- **4.4 CI/CD Pipeline** — Describe your continuous integration and deployment setup and tools (e.g., GitHub Actions, Jenkins).
- **4.5 Git Management (Version Controlling)** — Branching strategy, workflow, and collaboration rules.

5. Design

- **5.1 High-Level Design (HLD) – Component Level Design Following Patterns** — Major components, design patterns used (e.g., Factory, Observer), and their responsibilities.
- **5.2 Low-Level Design (LLD)** — Detailed design of individual modules, algorithms, data structures, and pseudocode.
- **5.3 Database Schema (ER Diagram + Schema)** — ER Diagram, table structures, keys, relationships, and normalization details.
- **5.4 GUI (Graphical User Interface) Design** — Wireframes, mockups, and final interface designs with explanations.
- **5.5 API Documentation** — List of all APIs with endpoints, methods (GET/POST), parameters, request/response examples.

6. Implementation / Construction

- **6.1 Development Environment and Tools** — IDE, version control, local and server setup.
- **6.2 Key Implementation Details / Modules Developed** — Important features/modules and how they were implemented.
- **6.3 Code Structure and Standards Followed** — Folder structure, naming conventions, coding standards, and best practices used.

7. Testing and Quality Assurance

- **7.1 Test Plan (Test Cases Design/Preparation)** — Overall testing strategy and approach.
- **7.2 Requirements/Specifications-Based System Level Test Cases** — Detailed test cases.
- **7.3 Traceability of Test Cases to Use Cases** — Traceability matrix.
- **7.4 Techniques Used for Test Generation** — Black-box, white-box, boundary value analysis, etc.
- **7.5 Unit Testing, Integration Testing** — Tools and results.
- **7.6 System Testing (UAT, PT, VT)** — User Acceptance Testing, Performance Testing, Volume Testing results.
- **7.7 Security Testing (VAPT)** — Vulnerability Assessment and Penetration Testing findings.
- **7.8 Assessment of the Goodness of the Test Suite** — Coverage, effectiveness, and limitations.

8. Deployment and Go-Live

- **8.1 Deployment Environment** — Server, cloud platform, configurations, and hosting details.
- **8.2 Deployment Process and CI/CD Implementation** — Step-by-step deployment process.
- **8.3 Go-Live Strategy and Post-Deployment Monitoring** — Rollout plan, training, monitoring, and support.

9. Sustainability Plan

- **9.1 Scalability** — How the system can handle growth in users/data.
- **9.2 Flexibility / Customization** — Features that allow future modifications.
- **9.3 Maintainability and Future Enhancements** — Maintenance plan and proposed future features.